RobotRover

System design, build notes and installation guide

Arduino UNO Q - Bluetooth LE - Android - Web Bluetooth - 8BitDo gamepad - LED matrix

What this document covers

- Sections 1-2: what the system is and the parts it is made of
- Section 3: how a command travels from a tap on the phone to a servo and the LED matrix
- Section 4: the seams, every place two components meet, what crosses, and how it fails
- Section 5: how the system was built, including the problems found along the way
- Section 6: how to install each part from scratch, and how to check it works
- Sections 7-10: what has been tested, security, troubleshooting, and a code map

Version 1.3 - November 13, 2026

Everything described here is open source and needs no accounts.

SECTION 1

At a glance

RobotRover is a small two-wheeled robot with a four-joint arm, driven from an Android phone, a desktop browser or a paired 8BitDo gamepad over Bluetooth. The brain is an **Arduino UNO Q**, a board with two processors: a Qualcomm chip running Debian Linux, and an STM32 microcontroller running the Arduino sketch. The board's 13 x 8 LED matrix shows what the robot is doing: an icon for each command, then a little face that reacts.

The design goal was a system that is **fully open source, needs no accounts, and works without an internet connection** once installed. Every tool used is either open source or downloaded directly from its publisher with checksum verification, and no component phones home.

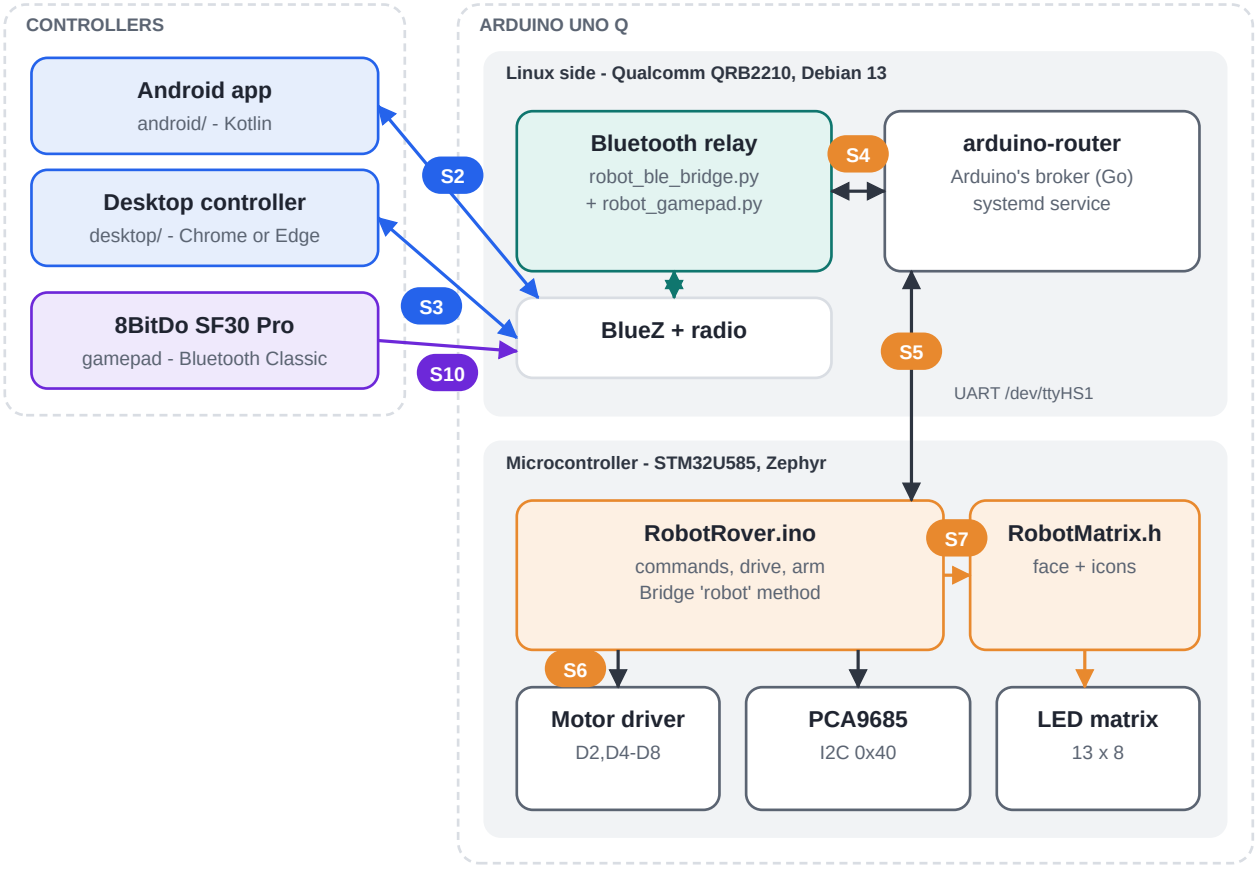


Figure 1. The components and the seams between them (S1, the command language, is shared by every seam and is not drawn). S8 (source-code mirrors) and S9 (build and deploy) are not runtime seams; Section 4 covers them. S10 is the optional gamepad, paired straight to the board.

Why there are so many pieces

The UNO Q's Bluetooth radio is wired to the Linux processor, not to the microcontroller that drives the motors. So a phone command has to land on the Linux side first and then be handed across to the microcontroller. Arduino provides that hand-off: a broker called `arduino-router` on Linux and a library called **Bridge** in the sketch, connected by an internal serial link. RobotRover adds a small Python program, the **relay**, that speaks

Bluetooth to the phone on one side and the router's protocol on the other. An 8BitDo SF30 Pro gamepad can also be paired straight to the board's radio; the relay turns its left stick, D-pad and buttons into the same commands, so the phone isn't needed to drive.

Component	Runs on	Language	Job
Android app	Phone (Android 8+)	Kotlin, no libraries	Drive pad, arm sliders, shows replies
Desktop controller	PC browser (Chrome/Edge)	HTML + JavaScript	Same controls, keyboard, console, demo mode, matrix preview
Gamepad (optional)	8BitDo SF30 Pro, paired to the board	-	Drives on the left stick, arm on the D-pad and triggers
Bluetooth relay	UNO Q, Linux side	Python 3.13	Bluetooth LE server; reads the gamepad; checks commands; forwards to the sketch; shuts the board down on request
arduino-router	UNO Q, Linux side	Go (Arduino's)	Message broker between Linux programs and the microcontroller
RobotRover sketch	UNO Q, microcontroller	C++ (Arduino, Zephyr)	Runs commands: wheels, servos, safety timeouts, battery gauge
RobotMatrix	UNO Q, microcontroller	C++ header	Draws icons and the face on the LED matrix

The phone app

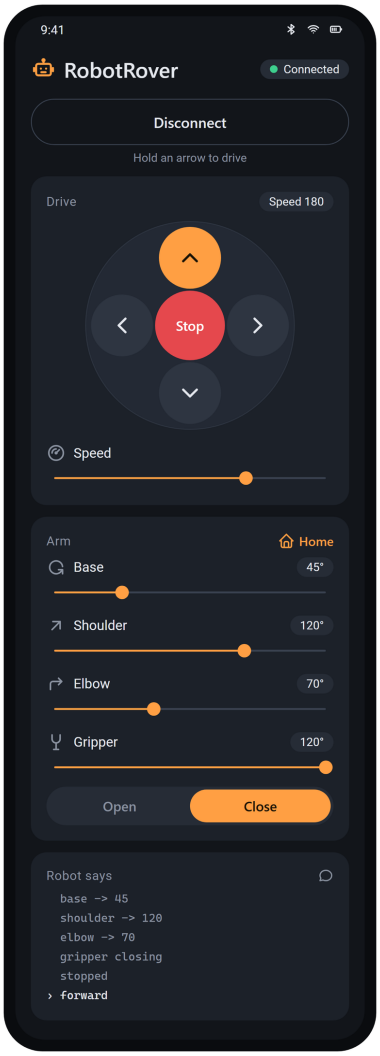


Figure 2 shows the Android app connected to the robot while the Forward arrow is held. From top to bottom:

- **Status pill:** grey when offline, yellow while searching or connecting, green when connected.
- **Connect / Disconnect** button, with a status line under it that explains problems (for example "Robot not found").
- **Drive card:** hold an arrow to drive; the held arrow lights orange. The red Stop sits in the middle. The pill and slider set the wheel speed.
- **Arm card:** one slider per joint with its angle in a pill. Home is in the corner. The Open / Close toggle follows the gripper slider.
- **Robot says:** the last six replies from the sketch, the newest one highlighted.
- **Sliders follow the robot:** the app asks for the positions when it connects and moves its sliders from the robot's replies, so they are right after the gamepad moved the arm or a move was refused.
- **Power off robot** (below Robot says, added after the mockup was drawn): asks for confirmation, then shuts the board down so the battery can be switched off safely.
- Controls are dimmed and disabled until the app is connected.

The desktop controller (desktop/index.html) has the same controls in a wider layout, plus keyboard driving, a command console, a demo mode and a live preview of the LED matrix.

Figure 2. The Android app, connected, with Forward held. Mockup drawn from the app's own layout, colors and icons; on a phone the screen scrolls.

SECTION 2

The hardware

Arduino UNO Q

- **Linux side:** Qualcomm QRB2210, Debian GNU/Linux 13 (trixie), Python 3.13.5, hostname `uno-q`, default user `arduino`. Owns the Wi-Fi and Bluetooth 5.1 radio and the USB-C port.
- **Microcontroller:** STM32U585 (Cortex-M33), running Arduino's Zephyr-based core (board package *Arduino Q Boards* 1.0.0, FQBN `arduino:zephyr:unoq`). Owns the UNO-style header pins and the LED matrix.
- **Link between them:** an internal UART, opened on Linux as `/dev/ttyHS1` by arduino-router.
- **I/O voltage:** the header pins are 3.3 V. The TB6612 / L298N motor drivers and the PCA9685 servo board accept 3.3 V logic.

Robot parts

Part	Connection	Notes
Left wheel motor	PWM D5, direction D7 / D8	Through a dual H-bridge (TB6612FNG recommended, L298N works)
Right wheel motor	PWM D6, direction D4 / D2	Flip LEFT_REVERSED / RIGHT_REVERSED in the sketch if a wheel runs backwards
Front ball caster	-	Passive; follows the wheels
Servo board (PCA9685)	I2C SDA / SCL, address 0x40	16-channel, 50 Hz. Gives the servos their own power
Base servo	PCA9685 channel 0	0 - 180 deg, home 90
Shoulder servo	PCA9685 channel 1	15 - 165 deg, home 90
Elbow servo	PCA9685 channel 2	15 - 165 deg, home 90
Gripper servo	PCA9685 channel 3	30 (open) - 120 (closed) deg, home 30. Goes limp after 3 s idle while open
LED matrix	Built into the UNO Q	13 x 8, 3-bit brightness (0 - 7)
Battery	2S Li-ion pack with BMS, through a main switch	7.4 V nominal, 8.4 V full; its protection cuts off at about 6.0 V
Servo regulator	Pack -> 5 V -> PCA9685 V+	Switching (buck or UBEC), 5 A or more (3 A for micro servos); must regulate down to 6 V in
Board regulator	Pack -> 5 V -> UNO Q 5V pin	Buck-boost, 3 A. Arduino's datasheet (rev 15 and later) allows powering the board here
Battery divider	20k from the pack to A0, 10k from A0 to GND	8.4 V reads 2.8 V at A0. Set BATTERY_SENSE in the sketch once wired

Power. One 2S Li-ion pack powers everything through a main switch: the motors straight from the pack, the servos and the board through their own 5 V regulators. Never power motors or servos from the UNO Q itself. All grounds connect together. The switch also cuts the board's Linux side, so use Power off in the app first.

Where everything connects

The pins RobotRover uses, colored by what they drive. The board is drawn to scale (68.6 x 53.3 mm) from Arduino's published top view of the UNO Q, with the header pins in their real positions.

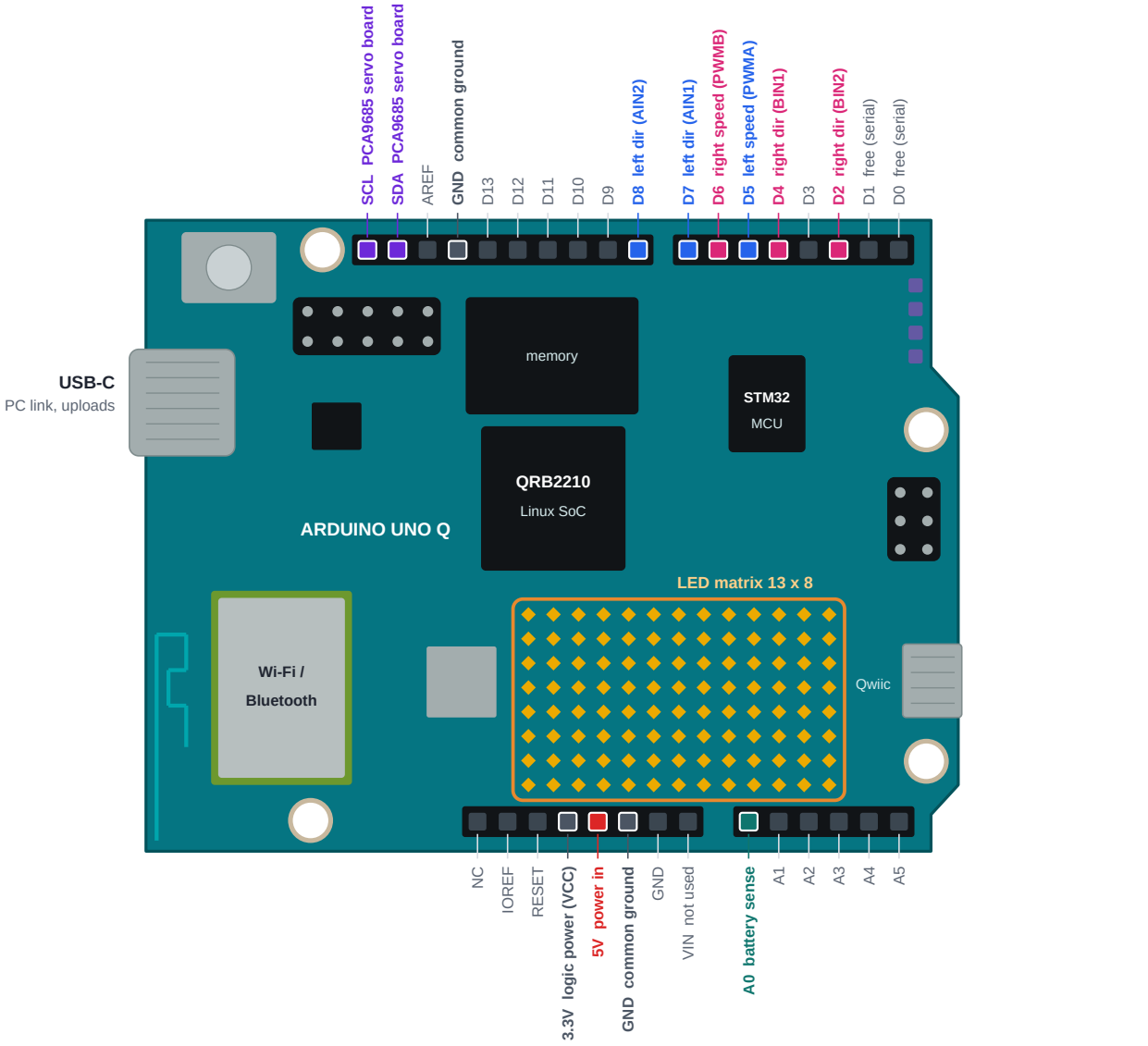


Figure 3. Arduino UNO Q pins used by RobotRover. Outline, headers, holes and major parts follow Arduino's top view; small parts are omitted and chip labels are approximate. Pins without a description are free. D5 and D6 must be PWM-capable pins.

Wiring

How the UNO Q, the TB6612FNG motor driver, the PCA9685 servo board, the motors, servos, regulators and battery connect. Wire colors match Figure 3: blue left wheel, pink right wheel, purple I2C and servo cables, orange 3.3 V logic, red power, teal battery sense, black ground.

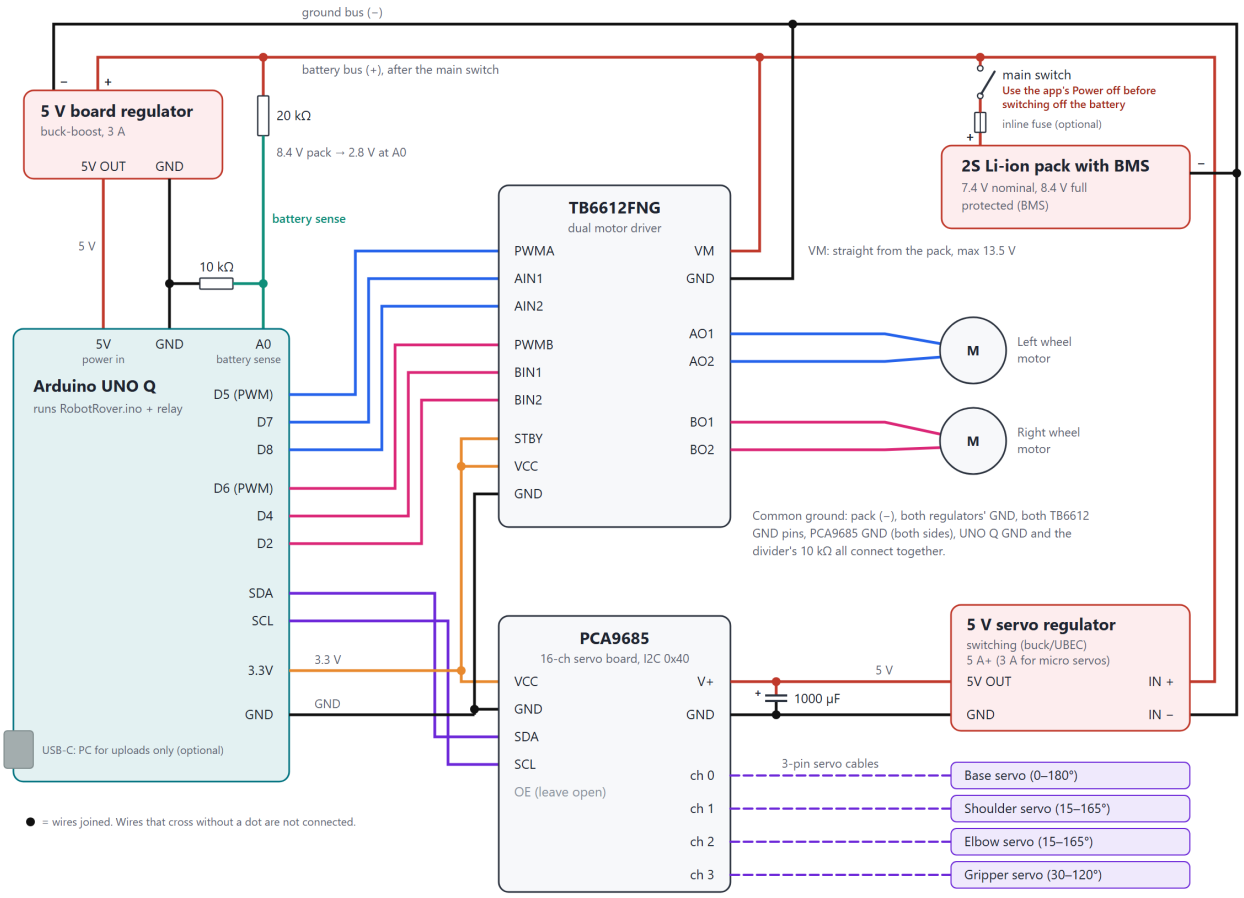


Figure 4. Wiring for RobotRover (schematic, not to scale). Pins match RobotRover.ino.

Rules that matter

- **One battery, three branches.** After the main switch the pack feeds the TB6612's VM directly (6 - 8.4 V; TB6612 maximum 13.5 V), the servo regulator (5 V into the PCA9685's V+) and the board regulator (5 V into the UNO Q's 5V pin).
- **Never power motors or servos from the UNO Q.** Its 5V pin is an input here, fed only by its own regulator.
- **Check polarity on the 5V pin.** Arduino's datasheet lists no reverse-polarity or over-voltage protection on it; keep it at 4.5 - 5.5 V. USB-C can stay plugged in for uploads: the board joins its inputs through diodes.
- **Don't feed the pack into VIN.** VIN needs 7 - 24 V, and a 2S pack spends its last stretch below 7 V.
- **The board regulator must be buck-boost** (step-up/step-down), so it keeps giving 5 V when motor surges sag the pack. The servo regulator must keep regulating down to a 6 V input.
- **Shut down before switching off.** The main switch also cuts the board's Linux side. Use Power off in the app and wait for the sleeping face on the matrix.
- **All grounds connect together:** UNO Q GND, both TB6612 GND pins, PCA9685 GND, both regulators' GND, the pack's (-) and the bottom of the battery divider.

- **Battery gauge:** 20k from the pack's + (after the switch, so it doesn't drain a parked robot) to A0, 10k from A0 to GND. Then set BATTERY_SENSE to true in the sketch and adjust BATT_CAL until `batt` matches a multimeter.
- **STBY must be tied high** (to 3.3 V), or the TB6612 ignores every command.
- **Logic is 3.3 V.** TB6612 VCC and PCA9685 VCC take 3.3 V from the board; the high-current power goes only to VM and V+.
- **Add a 470 - 1000 µF capacitor** across the servo board's V+ and GND to ride out current spikes.
- **Servo plugs:** signal (yellow or orange) toward the PCA9685's PWM row, brown or black toward GND.
- **Using an L298N instead?** ENA = D5, IN1 = D7, IN2 = D8, ENB = D6, IN3 = D4, IN4 = D2. Remove the ENA / ENB jumpers; motors go on OUT1 / OUT2 and OUT3 / OUT4.
- **First power-up:** wheels off the ground, then try `w 500` and `base 90` from the Serial Monitor or the desktop console.

SECTION 3

How a command travels

Following one command end to end is the quickest way to see how the parts fit. Here the user drags the Base slider on the phone to 45 degrees.

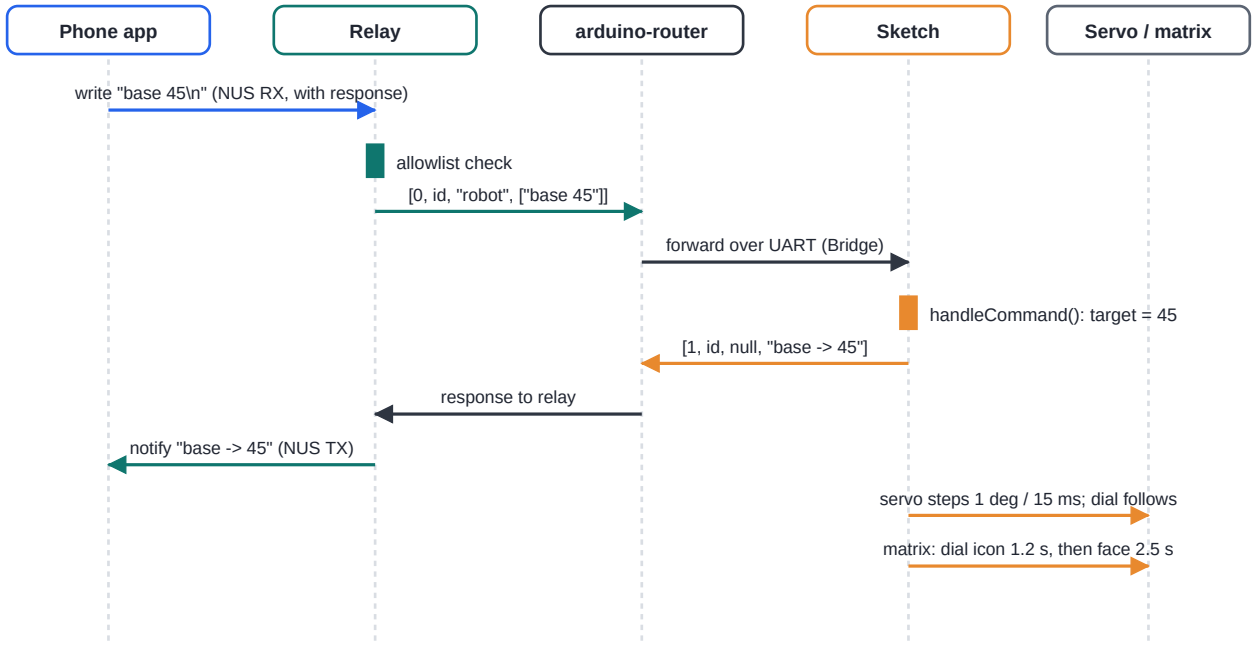
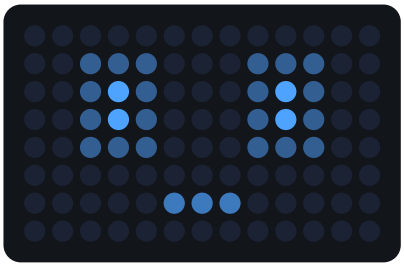


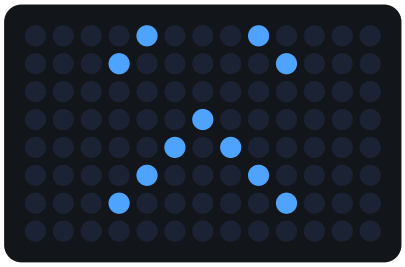
Figure 5. The life of "base 45". Each arrow crosses one seam from Section 4.

1. **Phone:** the slider sends `base 45` (at most every 100 ms while dragging, plus once on release). The app queues it, merging it with any older unsent `base` command, and writes it to the robot's Bluetooth characteristic.
2. **Relay:** receives the text, lower-cases it, checks it against an allowlist of robot commands, and calls the sketch's `robot` method through arduino-router's Unix socket.
3. **Router:** forwards the call over the internal UART to the microcontroller.
4. **Sketch:** Bridge runs the `robot` handler on its own thread, under a lock that keeps it from overlapping `loop()`. It parses the command, clamps 45 to the base's limits, sets the target angle, tells the LED matrix, and returns the reply `base -> 45`.
5. **Back out:** the reply returns through the router and relay and arrives at the phone as a Bluetooth notification, shown under "Robot says".
6. **Physically:** the servo moves toward 45 degrees one degree every 15 ms. The LED matrix shows a "B" dial whose needle follows the servo's actual position, then the face glances toward the side the base turned.

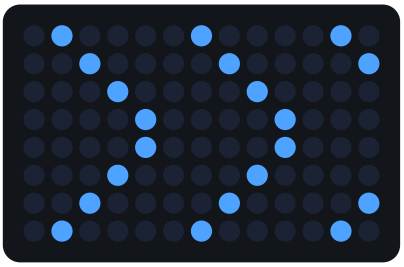
LED matrix frames



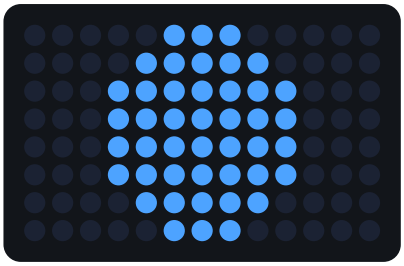
Idle face
Nothing happening; blinks every 4 s



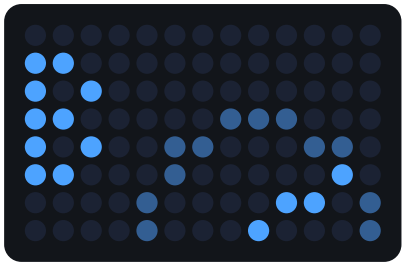
Forward
w held; the arrows stream upward



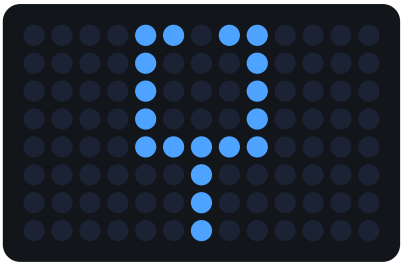
Spin right
d held; the arrows stream right



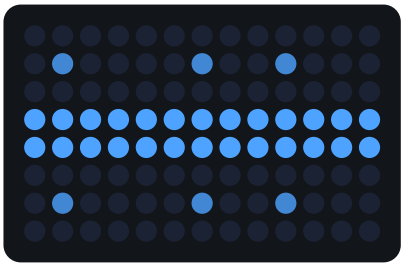
Stop
x; the sign pulses for 0.7 s



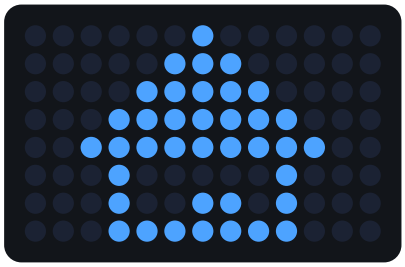
Base dial at 30 deg
The needle follows the real servo



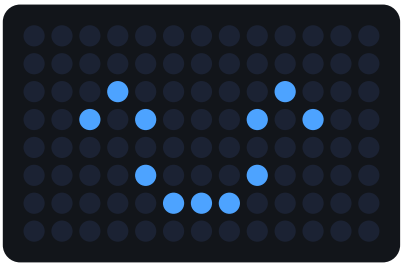
Gripper closed
The claw follows the real servo



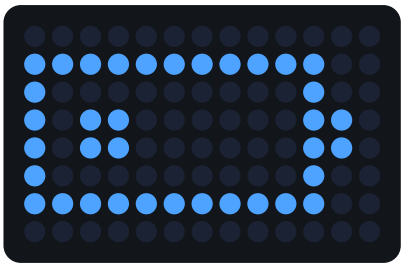
Speed 255
Bar length is the speed; dots stream



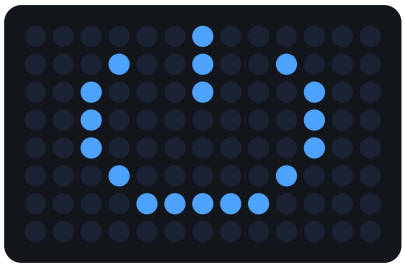
Home
home; shown for 1.2 s



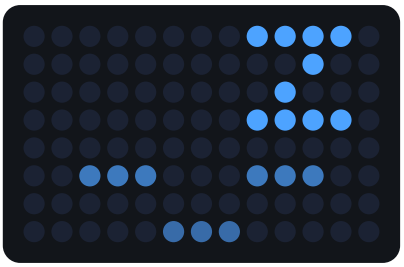
Happy face
For 2.5 s after home



Battery low
1 s of every 8 s; the bar is the charge



Powering off
Pulses while Linux shuts down



Safe to switch off
15 s after Power off

Figure 6. Real frames from RobotMatrix.h, drawn from the same pixel data the board shows. Each command shows its icon first; the face reacts afterwards. An empty battery shows the battery outline with no bar, blinking, in place of the faces.

Driving is a stream of short pulses

Holding a drive arrow does not send "drive forever". The app sends `w 400` (forward for 400 ms) every 150 ms. Each pulse restarts the sketch's 400 ms timer. Releasing the button sends `x` (stop). If anything between the phone and the motors fails (Bluetooth drops, the app is backgrounded, the relay restarts), the pulses stop arriving and the wheels stop on their own within 400 ms. The same rule is used by the desktop controller and by the gamepad (m L R 400 every 150 ms while the stick is pushed).

The sketch ramps each wheel up to speed over about 150 ms instead of jumping to full power, which cuts the current surge at every start. Slowing down, stopping and reversing are instant, so `x` still stops the robot at once. Repeated pulses set the same target, so holding a button doesn't restart the ramp.

SECTION 4

The seams

A seam is a place where two independently written components meet. Most bugs in a system like this live at the seams, so each one is described the same way: what crosses it, its exact format, which code owns each side, how it fails, and how to test it in isolation.

Seam	Between	Medium	Format
S1	Everything	-	The text command language
S2	Android app - relay	Bluetooth LE	Nordic UART Service, newline-terminated text
S3	Desktop controller - relay	Bluetooth LE (Web Bluetooth)	Same as S2
S4	Relay - arduino-router	Unix socket	MessagePack-RPC
S5	arduino-router - sketch	Internal UART	MessagePack-RPC via Bridge
S6	Sketch - motors and servos	GPIO / PWM / I2C	Pin levels, PCA9685 registers
S7	Sketch - LED matrix	Function calls	13 x 8 brightness frame
S8	C++ - JavaScript mirrors	Source code	Ported logic that must stay identical
S9	PC - devices (build and deploy)	USB	APK sideload, firmware upload, adb push
S10	Gamepad - relay	Bluetooth Classic (HID)	Linux input events, turned into S1 commands

S1 The command language

Every controller, the relay and the sketch share one tiny text protocol. It is the contract that lets each side be written, tested and replaced independently: a command typed into the Serial Monitor, the desktop console, the relay's `--send` option, or sent from the phone all behave the same. The one exception is `poweroff`: typed in the Serial Monitor it only puts the sketch in power-off mode, because only the relay can shut Linux down.

Command	Effect	Reply (examples)
w | s | a | d [ms]	Forward / back / spin left / spin right. Default 1000 ms; 0 = until stopped	forward, backward, spin left, spin right
m L R [ms]	Wheel speeds: left and right -255 to 255 (what the gamepad sends, mixed from its left stick). Default 1000 ms	wheels 180 -120
x or stop	Stop the wheels	stopped
speed N	Wheel speed 0 - 255 (default 180)	speed 200
base | shoulder | elbow | grip N	Move a joint to N degrees, clamped to its limits	base -> 45, base -> 180 (limit 0-180)
open | close | home	Gripper open / closed, whole arm to home	gripper opening, arm going home
pos	Report targets and speed	base 90 shoulder 90 elbow 90 gripper 30 speed 180

Command	Effect	Reply (examples)
batt	Report the battery voltage and state	battery 7.62 V ok, battery not connected
poweroff	Shut the board down (the relay does this; the sketch stops and locks out moves)	powering off: switch off when the face sleeps
help (Serial Monitor only)	Print the command list to the Serial Monitor	help printed to Serial Monitor

Rules: one command per line, lower case, single spaces, whole-number values. Replies are short single lines (they must fit in one Bluetooth packet; the relay truncates at 180 bytes). The relay only forwards commands matching this allowlist (from `board/robot_ble_bridge.py`); everything else is answered with `rejected: ...` and never reaches the sketch. When the battery is empty, or while powering off, every command that moves something is refused ("battery empty 6.5 V, charge me" or "powering off"). The pattern is one line in the code, and matches plain 0-9 digits only:

```
^(?:(?:w|a|s|d) \d{1,5}|w|a|s|d|x|stop|open|close|home|pos|batt|poweroff
  |(?:speed|base|shoulder|elbow|grip) \d{1,3}|m -?\d{1,3} -?\d{1,3}(?: \d{1,5})?)$
```

S2 Android app - relay (Bluetooth LE)

Crosses	Command text from the phone; reply text back.
Format	Nordic UART Service (NUS). Service 6e400001-b5a3-f393-e0a9-e50e24dcca9e. The phone writes to RX ...0002 (write with response), the relay notifies on TX ...0003. Each write is one command plus "\n"; the relay also splits a write on newlines.
Phone side	RobotBle.kt: scans for the service UUID or the name "RobotRover" (10 s timeout), connects with TRANSPORT_LE, requests a 185-byte MTU, discovers services, enables notifications by writing the 0x2902 descriptor, then sends through a one-at-a-time queue (max 20) that merges commands of the same kind (all drive commands are one kind).
Board side	robot_ble_bridge.py, built on bless 0.3.0 over BlueZ D-Bus. Advertises as "RobotRover". Commands go into an asyncio queue (max 50, oldest dropped) and are handled one at a time.
Permissions	Android 12+: BLUETOOTH_SCAN (neverForLocation) and BLUETOOTH_CONNECT. Android 8 - 11: location permission, and Location must be on. The app has no internet permission.
Fails how	"Robot not found": relay not running or Bluetooth off on the board. Connect then drop: radio range or the relay restarted. No replies but commands work: notifications not enabled.
Test it	Board: bluetoothctl show should list "Nordic UART Service" and ActiveInstances 1. Phone: the status pill turns green and "pos" (desktop console) gets a reply.

S3 Desktop controller - relay (Web Bluetooth)

Crosses	Exactly what S2 carries; the board cannot tell the PC and the phone apart.
Format	Same NUS service and characteristics. Uses navigator.bluetooth.requestDevice with a filter on the service UUID or name, writeValueWithResponse, and startNotifications.
PC side	desktop/index.html, served by desktop/serve.py on http://localhost:8765 bound to 127.0.0.1 (Web Bluetooth needs a secure origin; localhost counts). Chrome or Edge only.
Extras	Keyboard driving, a console for raw commands, demo mode (no hardware), and an LED matrix preview, and a Power off button. On connecting it asks for pos and from then on moves its sliders and preview from the robot's replies.
Fails how	"This browser can't use Bluetooth": wrong browser or the file was opened directly instead of through serve.py. Device list empty: PC Bluetooth off or relay not advertising.
Test it	Try demo mode first (exercises everything except Bluetooth), then Connect.

S4 Relay - arduino-router (MessagePack-RPC over a Unix socket)

Crosses	One RPC call per command: method "robot", one string parameter; the reply string back.
Format	MessagePack arrays on /var/run/arduino-router.sock (mode 666, so no root needed). Request [0, msgid, "robot", ["base 45"]]; response [1, msgid, error, result].
Relay side	RouterClient in robot_ble_bridge.py: a minimal client (about 60 lines) instead of Arduino's App Lab Python package, so the relay runs outside App Lab. 3 s timeout, one reconnect attempt on a dropped socket, ignores unrelated messages. The relay answers poweroff itself: it sends poweroff to the sketch, then runs systemctl halt (directly as root, or with sudo -n as the arduino user, which needs a sudoers rule). If that is refused it tells the phone why and sends poweroff 0 so the sketch carries on.
Router side	arduino-router (GPL-3.0) brokers calls in a star topology. Methods are registered by whoever provides them; here the sketch registers "robot".
Fails how	[2, 'method robot not available']: the sketch is not running, is an old version without Bridge.provide_safe("robot"), or is registered with an older router process (after a Linux-only reboot the microcontroller keeps running). The relay handles the last case: at startup, and whenever a command comes back "not available", it resets the microcontroller with Arduino's /opt/openocd/bin/arduino-reset.sh (GPIO, no sudo), at most once every 30 s. At startup it does so after 5 s of "not available" (20 s if the router doesn't answer at all). Don't drive while uploading a sketch, or this reset can interrupt the upload. Timeout: the sketch is stuck or rebooting. Connection refused: arduino-router is not running.
Test it	./run.sh --send pos on the board (no Bluetooth involved). Expected: base 90 shoulder 90 elbow 90 gripper 30 speed 180.

S5 arduino-router - sketch (Bridge over the internal UART)

Crosses	The forwarded RPC request and the sketch's reply.
Format	MessagePack-RPC over UART /dev/ttyHS1. Handled entirely by Arduino's Bridge library (Arduino_RouterBridge) in the sketch; RobotRover never touches the bytes.
Sketch side	setup() calls Bridge.begin(), Monitor.begin(), then Bridge.provide_safe("robot", bridgeCommand). "Safe" handlers are meant to run between loop() passes, but with Arduino_RPClite 0.3.x the Bridge's own thread runs them (measured on the board: a different thread from loop()). So runCommand() and loop() take turns through a Zephyr mutex, and command handlers only set targets; every pin and I2C write happens in loop(). loop() must stay fast and non-blocking.
Fails how	A burst of "invalid packet, expected array" in the router log right after the microcontroller resets is normal (Bridge flushes old data at startup). If the sketch blocks in loop(), every command times out at S4.
Test it	journalctl -u arduino-router shows the connection; S4's --send test proves S5 too.

S6 Sketch - motors and servos

Crosses	Electrical signals.
Wheels	Each motor: one PWM pin (analogWrite 0 - 255, 500 Hz) and two direction pins. IN1 high = forward, IN2 high = reverse, both low = coast. Commands set targets; updateDrive() ramps the output by 17 every 10 ms (0 - 255 in about 150 ms) and drops instantly when slowing, stopping or reversing. A timed drive stops itself when driveStopAt passes.
Servos	Adafruit PCA9685 library over I2C (address 0x40), 50 Hz, angle mapped to 500 - 2500 us. updateArm() moves each servo 1 degree every 15 ms toward its target, so motion is smooth and the power draw is spread out. At power-on the servos go to home one at a time, 250 ms apart. The open gripper goes limp after 3 s still (PCA9685 full-off) and wakes on its next command; the other joints always hold.
Battery	A0 through a 20k / 10k divider, 12-bit, sampled every 200 ms and smoothed over about 2 s. States: not connected (under 3 V), ok, low (under 7.0 V), empty (under 6.6 V for 3 s), with 0.2 V of hysteresis. Empty stops the wheels and refuses moves. Off until BATTERY_SENSE is true.
Fails how	"PCA9685 servo board not found" on the Serial Monitor at startup: wiring or power. Servos jitter or the board resets: regulator too small, or grounds not shared. batt says "not connected": pack switched off or divider not wired. Random battery readings: BATTERY_SENSE is true but nothing is on A0.
Test it	Serial Monitor commands (base 45, w 500, batt) with the robot on a stand.

S7 Sketch - LED matrix

Crosses	RobotMatrix::onCommand(cmd, hasValue, value) from handleCommand, RobotMatrix::update(angles, speed) from every loop(), setBattery(state, fill) after each battery reading, and setPoweringOff(on) for power-off.
Format	A 104-byte frame (13 columns x 8 rows, row-major), values 0 - 7, passed to Arduino_LED_Matrix::draw() after setGrayscaleBits(3). At most one frame every 40 ms.
Behavior	An icon for 0.6 - 1.5 s (refreshed while a command repeats), then a face reaction for 2.5 s, then the idle face (blinks every 4 s, glances every 9 s). The joint dial and claw draw the servos' current angles, not the targets. Battery low: a battery icon for 1 s of every 8 s. Empty: a blinking empty battery instead of the faces. Power off: a pulsing power symbol, then after 15 s a sleeping face, over everything else. A pos or help doesn't cut short an icon or reaction that is still showing.
Fails how	Matrix blank: sketch not running. All LEDs the same brightness or very dim: the grayscale setting; online reports about the UNO Q's brightness range conflict, so check this on the real board first.
Test it	Power the bare board: the idle face should appear within a second.

S8 Mirrors: C++ logic ported to JavaScript

The desktop controller's demo mode and its matrix preview contain line-for-line ports of sketch code. This is a source-code seam: nothing checks it automatically, so any change on one side must be copied to the other.

C++ original	JavaScript copy (desktop/index.html)	How it was verified
handleCommand() in RobotRover.ino	demoSketch(), battText()	Same replies for the full command table, including clamping, empty-battery and powering-off refusals
Relay allowlist (Python regex)	ALLOWED	Same pattern; demo rejects "help" exactly like the real relay
Relay's poweroff reply	demoReceive(), POWEROFF_REPLY	Same text; the demo adds "(demo)"
tankIcon() and the m command	tankIcon(), demoSketch()	Same arrow icons, clamping and replies
updateArm(): 1 degree / 15 ms	armClock interval	Dial and claw animate at the same speed
RobotMatrix.h (all drawing)	LM module	C++ compiled on the PC with g++ and stub libraries; its frames matched the JavaScript frames pixel for pixel for 38 frames, including the battery and power-off icons

S9 Build and deploy seams

Path	Tool	What crosses	Notes
PC -> phone	MTP file transfer	RobotRover.apk	Sideloaded from the Download folder. Debug-signed; Play Protect may warn.
PC -> microcontroller	arduino-cli (inside Arduino IDE 2) on COM13	Compiled sketch	Upload goes through the board's Linux side, which flashes the STM32 over SWD with OpenOCD.
PC -> board Linux	adb (platform-tools 37.0.1) over USB	Relay files and libraries	The board exposes an ADB interface on USB; no password needed for the arduino user.

Path	Tool	What crosses	Notes
Board boot	cron @reboot -> keepalive.sh	Starts the relay	Restarts it 3 s after any exit. cron skips the job while the arduino user's password is marked expired, as it is on a fresh board: set one with arduino-passwd. 20 s after starting, the relay restarts the sketch if it isn't registered with the new router.

S10 Gamepad - relay (Bluetooth Classic HID)

Crosses	Button and stick input from an 8BitDo SF30 Pro paired to the board. Nothing goes back to it.
Format	BlueZ's HID support turns the controller into a Linux input device (/dev/input/eventN). robot_gamepad.py reads its events with plain Python (no extra packages; the arduino user is in the input group). The button and stick codes are a table at the top of that file.
Board side	GamepadReader looks every 2 s for an input device named 8BitDo or SF30 with gamepad buttons. PadLogic turns its state into ordinary commands on the relay's queue, marked as coming from the pad; their replies aren't sent to the phone.
Driving	One stick: the relay mixes the left stick (up/down speed, left/right steering) into wheel speeds and sends m L R 400 every 150 ms while it is outside the 15% dead zone (sooner if a wheel's speed changes by more than 20). DRIVE_STYLE = "tank" drives each wheel from one stick instead. Releasing the stick, or losing the controller, sends x. While the pad drives, the phone's drive commands are ignored until 0.5 s after the pad's last one; stop from either always works.
Fails how	Not found: not paired, switched off, or not in D-input (Android) mode; Switch mode needs a driver this board doesn't have. "Has no sticks" in relay.log: wrong mode. A button does the wrong thing: this controller's codes differ from the table.
Test it	./run.sh --gamepad-test prints each button and stick as it changes.

Gamepad control	Does
Left stick	Drive: up / down is speed, left / right steers (spins in place), diagonals curve
Right stick	Not used
D-pad left / right	Base, 50 degrees a second while held
D-pad up / down	Shoulder up / down
L2 / R2	Elbow down / up
A / B	Gripper open / close
L / R	Top speed down / up by 25 (the same speed the phone's arrows use; the app's slider follows)
X	Battery icon on the matrix
Select	Stop
Start (tap, under 1 s)	Arm home
Hold Start + Select for 3 s	Power off

Values that must match across seams

If you change any of these, change every place listed.

Value	Where it lives
Service UUID 6e400001-..., RX ...0002, TX ...0003	robot_ble_bridge.py, RobotBle.kt, desktop/index.html
Device name "RobotRover"	robot_ble_bridge.py, RobotBle.kt, desktop/index.html
Bridge method name "robot"	RobotRover.ino (provide_safe), robot_ble_bridge.py (BRIDGE_METHOD)
Command allowlist	robot_ble_bridge.py (ALLOWED), desktop/index.html (ALLOWED)
Joint limits and home angles	RobotRover.ino (joints), activity_main.xml (slider min/max), MainActivity.kt (HOME_*), desktop/index.html (sliders, demo, HOME)
Gripper open 30 / closed 120	RobotRover.ino, RobotMatrix.h (claw), MainActivity.kt, desktop/index.html
Drive pulse 400 ms every 150 ms	MainActivity.kt, desktop/index.html, robot_gamepad.py (PULSE_MS, REPEAT_S)
Gamepad button and stick codes	robot_gamepad.py (BUTTONS, TRIGGER_AXES, LEFT_X, LEFT_Y, DPAD_X, DPAD_Y; RIGHT_Y for tank style); check with --gamepad-test
Reply texts that code reads	RobotRover.ino writes them. robot_gamepad.py, MainActivity.kt and desktop/index.html read "base -> N", the pos line, "speed N", "arm going home", "powering off" and "battery empty"; the desktop also reads the battery and poweroff replies
Controller name pattern 8bitdo|sf30	robot_gamepad.py (DEVICE_NAME) and pair-gamepad.sh
The list of drive commands	robot_ble_bridge.py (DRIVE_CMD) must list every command that drives the wheels
Reply length under 180 bytes	robot_ble_bridge.py (MAX_REPLY_BYTES), app MTU request (185)
Matrix shapes and timings	RobotMatrix.h and the LM module in desktop/index.html
Battery divider ratio (20k / 10k = 3.0)	The resistors and RobotRover.ino (BATT_DIVIDER)
Battery levels: empty 6.6 V, full 8.2 V	RobotRover.ino (BATT_*), desktop/index.html (BATT_EMPTY_V, BATT_FULL_V)
Power-off reply and the 15 s sleeping-face delay	robot_ble_bridge.py, RobotMatrix.h (SAFE_AFTER_MS), desktop/index.html (POWEROFF_REPLY, SAFE_AFTER_MS)
Power-off command /usr/bin/systemctl halt	robot_ble_bridge.py (POWEROFF_CMD) and the sudoers rule on the board

SECTION 5

How it was built

The project was built in one long session, in this order. The decisions and the problems found along the way are recorded here because they explain why things are the way they are.

Decisions

Decision	Why
Arduino IDE 2 / arduino-cli instead of App Lab	Open source, works offline once the board package is installed, no account.
Relay on the Linux side	The Bluetooth radio is only reachable from Linux; Arduino's Bridge is the supported way across.
Bluetooth LE with the Nordic UART Service	Widely supported text-over-BLE convention; easy to test with generic tools.
Own MessagePack-RPC client in the relay	Avoids depending on App Lab's Python package; about 60 lines.
Command allowlist in the relay	BLE here has no pairing, so the relay limits what a stranger could send.
Drive pulses instead of start/stop	Fails safe: losing the link stops the wheels within 400 ms.
PCA9685 for servos	Arduino's Servo library support on the UNO Q's Zephyr core could not be confirmed; the arm also needs separate servo power anyway.
Kotlin app with zero libraries	Small (about 860 KB), nothing to audit beyond the code, no internet permission.
Dark theme, round drive pad	Chosen from an interactive mockup before implementation.
Desktop controller via Web Bluetooth	Tests the same seam as the phone from a PC; demo mode needs no hardware.
"Face + icons" LED matrix design	Chosen from four animated concepts: clear about what is running, with personality.
SDK parts downloaded directly, not via Google's 'android' tool	The new tool collects usage data by default; direct downloads with checksums send nothing.
Relay installed offline as the normal user	The board had no internet and sudo needs a password; BlueZ's D-Bus policy and the router socket both allow a normal user.
One 2S battery for everything	One pack to charge. Motors run straight from it; the servos and the board get their own 5 V regulators, the board's a buck-boost so motor surges can't brown it out.
Commands set targets; loop() does the hardware	The Bridge runs commands on its own thread. A mutex plus this rule keeps pin and I2C writes from ever overlapping.
Battery cut-off at 6.6 V	Above the pack's own 6.0 V protection, so the robot stops cleanly before the pack cuts all power.
Power off through the relay	The main switch also cuts Linux. A button in the app shuts it down first, with no shell access needed; the sudoers rule allows only that one command.
Halt, not poweroff	On USB power this board starts again within seconds of a poweroff, but stays down about 45 s after a halt, with the microcontroller still showing the matrix.

Decision	Why
Ramp the wheels, stagger the servos, relax the open gripper	Fewer current spikes on the shared pack. The base stays powered because the arm can be knocked while driving.
Gamepad paired to the board, read by the relay	Drives without a phone, and the phone still works. Going through the relay's queue reuses its allowlist, power-off and restart handling.
Wheel-speed command m L R	Lets the relay mix one stick into smooth curves (or drive tank style); the existing ramp works per wheel.
Gamepad owns the wheels while driving	Phone and pad pulses would otherwise fight every 150 ms. Stop always works.

Problems found and fixed

Problem	Cause	Fix
Phone could not connect	Relay never installed; board had no internet for setup.sh	Packages bundled on the PC, pushed with adb, run as user 'arduino', started by cron
Relay got "method robot not available"	Board still had the first sketch version, from before the Bluetooth work	Compiled and uploaded the current sketch with arduino-cli
pip downloaded the wrong packages	Dependency markers were evaluated for Windows, not the board	Downloaded exactly five pinned packages with --no-deps
sdkmanager no longer manages licenses	Google replaced it with the 'android' CLI, which collects usage data	Read the license, user accepted it, downloaded packages directly
Keyboard driving did not start in a test	Handler assumed every key event came from an element	Guarded the check; real keypresses and synthetic ones now both work
Matrix preview stayed blank in the in-app browser	requestAnimationFrame does not fire there	Switched to a 20 ms timer (also keeps running when the window is covered)
Relay start command killed itself	pkill -f matched its own command line	Used the pattern [r]obot_ble_bridge.py
Commands could run at the same time as loop()	Bridge (Arduino_RPClite 0.3.x) runs "safe" handlers on its own thread, not between loop() passes	A Zephyr mutex around commands and loop(); confirmed with a thread-ID test build
Relay never started after a reboot	The arduino user's password is marked expired, so cron refuses its @reboot job	Set a password with Arduino's arduino-passwd tool (Section 6, B4)
Arm jumped back when the D-pad was used after the phone	The gamepad only knew the angles from its own commands	The relay passes every sketch reply to the gamepad logic; the apps follow the replies too
Robot stopped while Forward was still held	The app's scrolling page took over the touch when the thumb drifted	The drive buttons keep the touch (requestDisallowInterceptTouchEvent)
Sketch unreachable after a reboot	A Linux-only reboot doesn't reset the microcontroller; it stays registered with the old router	The relay resets the microcontroller through arduino-reset.sh when the sketch doesn't answer

Tool and package versions

Area	Versions
Microcontroller	Arduino Q Boards (arduino:zephyr) 1.0.0, arduino-cli 1.5.1, Arduino_RouterBridge, Adafruit PWM Servo Driver Library, Arduino_LED_Matrix (bundled)
Board Linux	Debian 13, Python 3.13.5, BlueZ; relay packages bless 0.3.0, bleak 3.0.2, dbus-fast 5.2.0, dbus-next 0.2.3, msgpack 1.2.3
Android	Gradle 8.9, Android Gradle Plugin 8.7.3, Kotlin 2.0.21, compileSdk / targetSdk 35, minSdk 26, build-tools 34.0.0, JDK 21
Tools on the PC	Android SDK in D:\Android\Sdk (platform 35 r02, build-tools 34.0.0, platform-tools 37.0.1), Python 3.12 for the desktop server

SECTION 6

Installation

Install in this order: the sketch, then the relay, then a controller. Each step ends with a check that proves its seam works before moving on.

Prerequisites

Needed for	What
Sketch	Arduino IDE 2 (or arduino-cli) with the Arduino Q Boards package and the two libraries
Relay	USB cable and adb (Android platform-tools), or SSH and internet on the board
Android app	JDK 17 or newer, the Android SDK parts below (or Android Studio), a USB cable
Desktop controller	Python 3 and Chrome or Edge, on a PC with Bluetooth

A. The sketch (microcontroller)

1. In Arduino IDE 2: **Boards Manager** - install **Arduino Q Boards**. **Library Manager** - install **Arduino_RouterBridge** and **Adafruit PWM Servo Driver Library**.
2. Open RobotRover.ino (RobotMatrix.h must sit in the same folder), select **Arduino UNO Q**, and upload. Or from a terminal in the project folder:

```
arduino-cli compile -b arduino:zephyr:unoq .
arduino-cli upload  -b arduino:zephyr:unoq -p COM13 .     # use your board's port
```

Check: the LED matrix shows the idle face. In the Serial Monitor (line ending "New Line"), `pos` answers with the joint angles.

Battery gauge: once the 20k / 10k divider is wired to A0, set `BATTERY_SENSE = true` in RobotRover.ino and upload again. Compare `batt` with a multimeter on the pack and adjust `BATT_CAL` until they agree.

B. The Bluetooth relay (board Linux side)

Option 1: offline over USB (how the current board is set up)

1. Install Android platform-tools to get adb, and check the board is listed with `adb devices`.
2. On the PC, download the five packages built for the board (aarch64, Python 3.13) and unpack them into a `lib` folder next to the relay files:

```
python -m pip download --no-deps --only-binary=:all: --dest wheels \
    --platform manylinux2014_aarch64 --platform manylinux_2_28_aarch64 \
    --python-version 3.13 --implementation cp --abi cp313 --abi none \
    bless==0.3.0 bleak==3.0.2 dbus-fast==5.2.0 dbus-next==0.2.3 msgpack==1.2.3
# unzip every .whl in wheels/ into robot-ble/lib/  (a wheel is a zip file)
```

3. Copy the libraries and the five board files to `/home/arduino/robot-ble/` (from the project folder), then make the scripts executable and start the relay at boot:

```
adb shell "mkdir -p ~/robot-ble"
adb push robot-ble/lib /home/arduino/robot-ble/
adb push board/robot_ble_bridge.py board/robot_gamepad.py /home/arduino/robot-ble/
adb push board/run.sh board/keepalive.sh board/pair-gamepad.sh /home/arduino/robot-ble/
adb shell "chmod +x ~/robot-ble/*.sh"
# note: the next line replaces the arduino user's crontab (it is empty on a fresh board)
adb shell "echo '@reboot /home/arduino/robot-ble/keepalive.sh' | crontab -"
adb shell "cd ~/robot-ble && setsid nohup ./keepalive.sh >/dev/null 2>&1 </dev/null &"
```

4. **Set a password for the arduino user.** A fresh board's password is marked expired, and cron skips the @reboot job until it is changed. In `adb shell` on the board, run Arduino's own tool `sudo /usr/local/bin/arduino-passwd`, type the new password and press Enter. It doesn't ask for the old password or a sudo password. `chage -l arduino` should then say the password never expires.
5. **Allow Power off** (optional, needed for the app's Power off button). In `adb shell` on the board, add a sudoers rule that allows only that one command (sudo asks for the password from step 4):

```
echo 'arduino ALL=(root) NOPASSWD: /usr/bin/systemctl halt' \
    | sudo tee /etc/sudoers.d/robot-poweroff
sudo chmod 440 /etc/sudoers.d/robot-poweroff && sudo visudo -c
```

Check S4/S5: `adb shell "cd ~/robot-ble && ./run.sh --send pos"` prints the joint angles. **Check S2:** `adb shell "bluetoothctl show"` lists the Nordic UART Service, and `~/robot-ble/relay.log` says "Advertising as 'RobotRover'".

Option 2: standard install (board has internet)

Copy the `board` folder to the board (scp or adb push) and run `bash setup.sh`. It installs packages with apt and pip, puts the relay in `/opt/robot-ble` and runs it as a root systemd service (`robot-ble-bridge`). It asks for the sudo password, so set one first (Option 1, step 4). Running as root, the relay can power the board off without the sudoers rule.

With this install, the commands elsewhere in this guide change: instead of `cd ~/robot-ble && ./run.sh ...` use `sudo /opt/robot-ble/venv/bin/python /opt/robot-ble/robot_ble_bridge.py ...` (--send pos, --gamepad-test, --reset-mcu); pair the gamepad with `/opt/robot-ble/pair-gamepad.sh`; read the log with `journalctl -u robot-ble-bridge` instead of relay.log; and restart the relay with `sudo systemctl restart robot-ble-bridge`.

C. The Android app

1. Get the SDK parts: either install Android Studio, or download **platform 35** and **build-tools 34.0.0** from Google's repository (the Android SDK License Agreement applies) into a folder such as D:\Android\Sdk, and point `android/local.properties` at it (`sdk.dir=D\:\\Android\\Sdk`).
2. Build with a JDK 17+ as JAVA_HOME:

```
cd android
gradlew.bat assembleDebug          # output: app/build/outputs/apk/debug/RobotRover.apk
```

3. Plug the phone in with **File transfer** on and copy the APK to its **Download** folder.
4. On the phone: **Files - Downloads**, tap the APK, allow **Install unknown apps** for Files, tap **Install**, and choose **Install anyway** if Play Protect warns.

Check: open RobotRover, tap **Connect to robot**, allow the Bluetooth permission. The pill turns green; Home or Open/Close produce replies and matrix icons.

D. The desktop controller

```
python desktop/serve.py            # opens http://localhost:8765 in your browser
```

Use Chrome or Edge. **Try demo** works without hardware; **Connect to robot** opens the browser's Bluetooth device picker. Keyboard: W/A/S/D or arrows to drive, Space stop, O/C/H gripper and home.

E. The gamepad (optional)

1. Put the 8BitDo SF30 Pro in **D-input (Android) mode**: hold **Start + B** to switch it on. Then hold **Pair** for 3 s until the lights blink quickly.
2. On the board, run the pairing script. It looks for the controller for up to 2 minutes, then pairs, trusts and connects it:

```
adb shell "cd ~/robot-ble && ./pair-gamepad.sh"
```

3. Check the map: `adb shell "cd ~/robot-ble && ./run.sh --gamepad-test"` prints each button and stick as you press it. If a label is wrong, edit BUTTONS at the top of robot_gamepad.py, push it again and stop the running relay so it restarts with the change (`pkill -f '[r]obot_ble_bridge.py'`).

Check: relay.log says "gamepad connected", and pushing the left stick forward shows the forward arrows on the matrix. Afterwards, switching the controller on in D-input mode connects it again by itself.

Verification checklist

#	Check	Proves
1	Matrix shows the idle face after power-on	Sketch running (S7)
2	Serial Monitor: pos answers	Sketch command handling (S1)
3	./run.sh --send pos answers on the board	Relay - router - sketch (S4, S5)
4	bluetoothctl show lists the Nordic UART Service	Relay advertising (S2)
5	Phone connects and Home gets "arm going home"	Whole chain (S2 to S7)
6	Hold an arrow, then turn the phone's Bluetooth off: wheels stop within 0.4 s	Drive safety
7	Reboot the board; phone reconnects after about a minute (needs the password set, step B4)	Boot autostart (S9)
8	batt answers ("battery sensing off" until the divider is wired)	Battery gauge (S6)
9	Power off in the app: power symbol, then the sleeping face; Linux shuts down	Safe shutdown (S4, S7)
10	Gamepad: left stick forward drives; letting go stops; switching it off mid-drive stops	Gamepad (S10)

SECTION 7

What has been tested

Verified	How
Phone app connects to the board over Bluetooth	On the real phone (moto g stylus 5G 2023) and board
Relay - router - sketch round trip	./run.sh --send pos and home on the board
Relay advertising and autostart setup	bluetoothctl show; keep-alive running; crontab entry present
Sketch compiles for the UNO Q and uploads	arduino-cli 1.5.1, and 0.32.2 (bundled with Arduino IDE 2); Arduino Q Boards 1.0.0
Android app builds cleanly	Gradle 8.9 / AGP 8.7.3, no compiler warnings; APK permissions inspected with aapt2
RobotMatrix.h drawing logic	Compiled on the PC with g++ -Wall and stubs; 38 frames checked
C++ and JavaScript matrix match	Pixel-identical frames for the same inputs, including battery and power-off icons
Desktop demo mode, keyboard and console	Automated checks in a browser
Desktop demo: batt, Power off, refused moves	Script-driven checks in Node with a stub page
Relay protocol code and allowlist	Unit test against a fake router, including the error path
Commands run on a different thread from loop()	Thread-ID test build on the board
The command lock under load	300 commands in 2.6 s through the router on the board, no lost replies
batt, poweroff, poweroff 0 and the move lockout	On the board through arduino-router
Power off refused without the sudoers rule	On the board: the phone is told why, the robot carries on
Power off all the way down	On the board (USB power): Linux shut down cleanly; after a halt the matrix showed the sleeping face and the board stayed down about 45 s
Wheel ramp, gripper relax and battery states	Unit tests on the PC with g++
Wheel-speed command m	On the board: m 200 200 300, m -120 150, clamping, a malformed m rejected
Gamepad logic	Unit tests on the PC: one-stick and tank drive, dead zone, pulses, arm steps, clamped replies, buttons, power-off hold, disconnect, following the phone's changes, pausing after a refused move
Relay queue	Test on the PC with a slow fake router: stale drive pulses dropped, stop and other commands kept, non-ASCII digits rejected
Slider sync on the desktop page	Script-driven check in Node: pos, joint and speed replies move the sliders; a dragged slider is left alone
Real SF30 Pro: pairing, reconnecting, D-pad, Start + Select	On the board: arm targets and the matrix followed the D-pad; the hold shut the board down
Gamepad end to end	On the board with a virtual SF30 Pro (Linux uinput): the relay found it, drove, moved the base, opened and closed, changed speed, went home, and stopped when it vanished

Verified	How
Relay autostart and sketch restart after a reboot	sudo reboot on the board: relay started from cron, reset the microcontroller, sketch answered 31 s after boot

Not yet verified	Notes
Motors, servos and the PCA9685 in the robot	Not wired yet; test on a stand first
Battery gauge on a real pack	Divider not wired yet; calibrate against a multimeter
Power off on battery power	On USB power the board starts again about 45 s after Power off; on battery it may stay off
Gripper relaxing saves current	Some digital servos keep holding with no pulses
LED matrix brightness levels on the real board	Conflicting reports online about the grayscale range
Desktop controller over real Bluetooth	Demo mode only so far
Wheels stop when the link drops	Logic is in place; check with item 6 of the checklist
Driving with the real SF30 Pro	Its axes match the map. Driving with the left stick, and the A / B / X / L / R labels (--gamepad-test), are still to be tried
App changes in 1.3 on a real phone	Hold-to-drive with a drifting thumb, sliders following the robot, quicker stop
Desktop: Cancel while connecting	Fixed from reading the code; needs real Bluetooth to confirm
Phone and gamepad at the same time	The drive hand-over is in place but untested with a real phone
Cold start from the battery switch	Both processors start together; the relay's 20 s wait should cover the sketch's start-up, and it resets the microcontroller if not

SECTION 8

Security and privacy

- **No accounts anywhere.** No Arduino Cloud, no Google sign-in, no App Lab account.
- **Nothing phones home.** The Android app has no internet permission. The desktop page loads nothing from the internet and its server only listens on 127.0.0.1. Google's new 'android' SDK tool (which collects usage data by default) was not used for installs; its two help commands were run once before that was known.
- **Bluetooth is open.** There is no pairing or PIN, so anyone nearby with a Bluetooth app could connect while the relay runs. The relay's allowlist means they can only send robot commands, never anything to the Linux system. Stop the relay when not in use, or add a PIN in the relay as a future step.
- **Power off is open too.** Anyone who can connect can shut the board down, the same exposure as driving it.
- **Least privilege.** With the offline install the relay runs as the normal 'arduino' user, not root. Its one extra right is the sudoers rule for `systemctl halt`, which allows that command and nothing else. The standard install (setup.sh) runs the relay as a root systemd service instead.
- **Drive time 0 means "until stopped".** A command such as `w 0` keeps the wheels turning even if the link then drops. The apps and the gamepad never send it; the desktop console and any other Bluetooth client can.
- **Gamepad.** The relay treats any paired input device named like an 8BitDo controller as the gamepad. pair-gamepad.sh switches pairing off again when it finishes.
- **Downloads were verified.** Every SDK and Gradle download was checked against the publisher's SHA-1 or SHA-256; Python packages came from pypi.org over HTTPS with pip's hash checks.
- **License.** Building the Android app uses Google's Android SDK, under the Android SDK License Agreement (accepted for this project). The RobotRover code itself does not depend on it at runtime.

SECTION 9

Troubleshooting by seam

Symptom	Seam	Check
App: "Robot not found"	S2	Relay running? tail ~/robot-ble/relay.log; bluetoothctl show (Powered: yes). Android 11 or older: Location on.
Replies say "robot restarting, try again in a few seconds"	S4/S5	The relay found the sketch unregistered (usually after a reboot) and is restarting it. If it keeps happening, re-upload RobotRover.ino.
run.sh --send says 'robot' failed ... not available	S4/S5	Same cause: ./run.sh --reset-mcu restarts the sketch. If that doesn't help, re-upload RobotRover.ino.
Replies say error: no answer from 'robot'	S5	Sketch stuck or rebooting; check the Serial Monitor.
Replies say rejected: ...	S1	The command is not in the allowlist (expected for help or typos).
Matrix blank	S7	Sketch not running; re-upload.
"PCA9685 servo board not found"	S6	SDA/SCL wiring, servo board power, shared ground.
A wheel runs backwards	S6	Flip LEFT_REVERSED or RIGHT_REVERSED.
Desktop: "This browser can't use Bluetooth"	S3	Use Chrome or Edge, opened through serve.py.
Relay not running after reboot	S9	crontab -l shows the @reboot line; relay.log shows "relay starting". journalctl -u cron saying "expired password": set one with arduino-passwd.
Replies say battery empty ... charge me	S6	Charge the pack. Moves unlock once it reads 6.8 V or more.
batt says battery not connected	S6	Pack switched off, or the divider isn't wired to A0.
Power off says not allowed	S4	Add the sudoers rule (Section 6, B5).
Gamepad does nothing	S10	relay.log should say "gamepad connected". If not: switch it on in D-input mode (Start + B), or pair it again with pair-gamepad.sh.
Gamepad buttons do the wrong thing	S10	Run ./run.sh --gamepad-test and fix BUTTONS in robot_gamepad.py.
Phone's drive buttons don't work	S10	The gamepad is driving; the phone gets the wheels about 0.5 s after the left stick is released.
Board resets when the motors or servos start	S6	Board regulator not buck-boost or too small, servo regulator too small, or grounds not shared.

SECTION 10

Code map and file map

Where the code lives, what each part is written in, and what it builds. Line counts are rounded and were taken at release 1.3, so they drift as the code changes.

Lines per folder

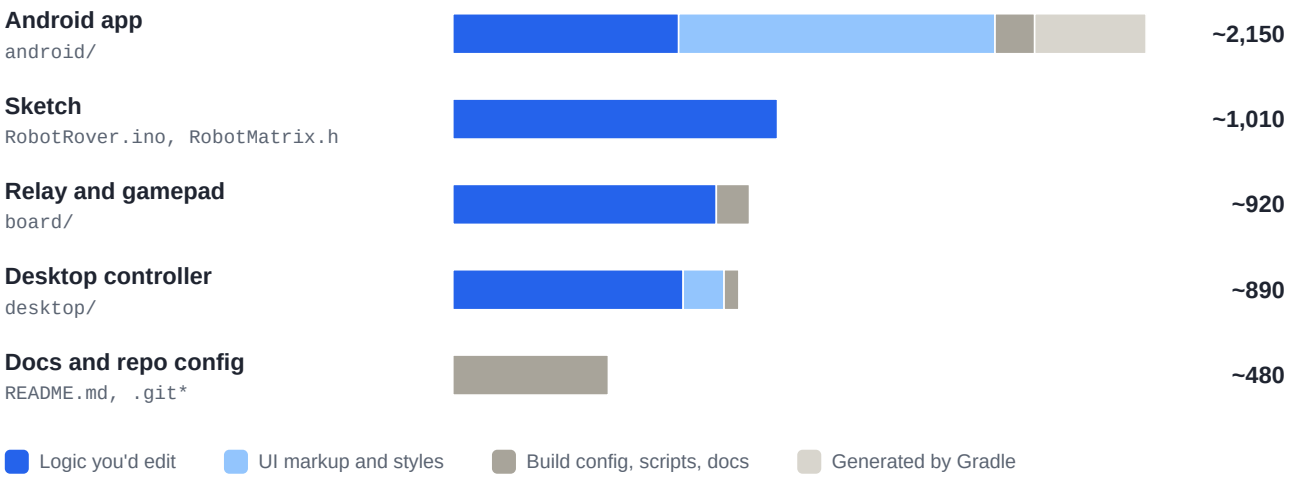


Figure 7. About 5,450 lines of text files in git at release 1.3 (about 4,000 at the first release). Images, including the SVG sources, and the PDF are not counted, and neither is docs/build_pdf.py.

The Android folder is about 40% of the line count, but mostly screens and icons. The sketch has the most logic: about 1,010 lines of C++, then 820 of Python in the relay and gamepad support, 715 of JavaScript in the desktop page, and 700 of Kotlin in the app.

What each part builds

Part	Logic	Builds with	Produces	Runs on
Sketch	~1,010 lines C++	arduino-cli 1.5.1 or Arduino IDE 2, board arduino:zephyr:unoq	Firmware, about 126 KB (16% of the sketch's flash area)	UNO Q's STM32U585
Android app	~700 lines Kotlin	Gradle 8.9, Android Gradle Plugin 8.7.3, Kotlin 2.0.21	RobotRover.apk, about 860 KB	Android 8 or newer
Desktop controller	~715 lines JavaScript	No build step; desktop/serve.py serves it	A page at localhost:8765	Chrome or Edge on a PC
Bluetooth relay	~820 lines Python	No build step; copied to the board with adb	A Bluetooth service named RobotRover	UNO Q's Linux side
This document	~1,530 lines Python	python docs/build_pdf.py (needs reportlab)	This PDF	Any PC with Python 3

Only the sketch and the Android app have a real build step. The relay and the desktop page run as plain source files.

File map

Path	What it is	Size
RobotRover.ino	The sketch: commands, wheels, servos, battery gauge, wheel-speed command (m), Bridge registration	~600 lines
RobotMatrix.h	LED matrix icons and face (part of the sketch)	~410 lines
board/robot_ble_bridge.py	The Bluetooth relay: MessagePack-RPC client, allowlist, power-off, sketch restart	~420 lines
board/robot_gamepad.py	Gamepad support: finds and reads the 8BitDo, turns it into commands, --gamepad-test	~400 lines
board/pair-gamepad.sh	Pairs, trusts and connects the gamepad with bluetoothctl	~35 lines
board/run.sh, board/keepalive.sh	Run the relay with bundled libraries; keep it running (offline install)	~15 lines
board/setup.sh, board/robot-ble-bridge.service	Standard install with apt/pip and a systemd service	~50 lines
android/	Android app (Kotlin). Gradle wrapper included; local.properties is per-PC and not in git	~2,150 lines
android/app/src/main/java/.../RobotBle.kt	Bluetooth LE client: scan, connect, queue, notifications	~310 lines
android/app/src/main/java/.../MainActivity.kt	Screen logic: drive pulses, sliders that follow the robot, status, log, power off	~390 lines
android/app/src/main/res/	Layout, theme, colors, icons, launcher icon (XML)	~980 lines
desktop/index.html	Desktop controller, demo model and matrix preview (ports of sketch code)	~840 lines
desktop/serve.py	Local web server on 127.0.0.1:8765	~50 lines
docs/build_pdf.py	Generates this document; draws every figure except the app and wiring images	~1,530 lines
README.md	Quick start and reference	-
RobotRover_System_Guide.pdf	This document	-
images/	App mockup, board pinout and wiring images (used in the README and this document), with the SVG sources of the pinout and the wiring diagram, and the README's GIF of the board	-

Not in the repository. Build outputs (the APK and the firmware); the five Python packages bundled for the relay on the board (Section 6 has the download command); Arduino's arduino-router, which comes with the board; android/local.properties, which is specific to each PC; and the HTML page used to render images/app_mockup.png.